Modular proofs and programs in Lean

Internship report
Arthur Adjedj
M2 MPRI, Université Paris-Saclay, ENS Paris-Saclay
Supervised by Yannick Forster in the Cambium team, INRIA Paris,

Interactive theorem provers (ITPs) do not easily allow users to adapt and extend existing codebases without either changing the original code or duplicating it. The pattern of extending definitions occurs particularly often in programming language theory, type theory, and program verification, leading to a lot of code duplication and high maintenance burden. Existing approaches to this problem either require encoding data-types as complicated expressions—obfuscating the development— or rely on custom intermediate representations of syntax via meta-programming facilities, which were underdeveloped in most ITPs until recently.
We develop **Gemel**[1] : a tool implemented in Lean 4 to produce modular code, leveraging the strengths of dependent types and modern meta-programming techniques

1 Summary

1.1 General context

Interactive Theorem Provers (ITPs), also known as proof assistants, are tools which allow for the development and mechanical verification of formal proofs. ITPs can be used to provide a very high level of guarantees for software (in particular the complete absence of bugs), and several projects make use of proof assistants to verify real-world software, such as the CompCert C compiler (Leroy [2009]), the sel4 operating systems kernel (Lyons et al. [2025]), and AWS' authorization policy language, Cedar (AWS [2026]). They have also been used successfully to formalize landmark theorems in mathematics such as the Four Colour Theorem or the Feit Thompson theorem, and the use of proof assistants is on the rise in mathematics departments, with big mathematical developments like Lean's Mathlib library (The mathlib Community [2020]) getting more and more public attention.

Like other programming languages, ITPs fall victim to the the fact that reusing definitions can be non-trivial. This is generally referred to as the **expression problem** (Wadler [1998]):

> "The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, *without recompiling existing code*."

In fact, reusing and adapting existing data structures and programs modularly is a challenge for which few solutions have been produced over the years in industrial

[1] Pronounced gem·el, describes pairs of trees that have undergone inosculation, i.e the act of connecting and merging with one another.

programming languages (Oliveira and Cook [2012]). It is even worse for ITPs, where in addition to programs, one needs to adapt proofs and dependently typed programs as well.

1.2 Research problem

The original expression problem from 1998 however does not exactly fit the constraints of modern ITPs, or even arguably that of programming languages, in particular with regards to avoiding recompiling existing code. Indeed, computers have since become fast enough, and memory storage big enough, that the constraints of compilation time or executable sizes are not central issues anymore. Our goal is instead to allow, in a proof-assistant, for defining datatypes as extensions of others through the addition of new constructors, as well as allow for extending theorems of definitions which manipulates datatypes such that the new ones may instead manipulate extensions of said datatypes.

1.3 Your contribution

We present **Gemel**, a new library, written in the Lean 4 proof assistant, which exposes new syntax for users to extend previous data-types and declarations modularly, using meta-programming techniques. This in particular allows one to extend a previous formalization that was not intended for such uses.

1.4 Arguments supporting its validity

To demonstrate the capacitie of our system, we present two case-studies implemented using our tool. The first takes an existing, independent formalization of the Simply-Typed Lambda Calculus (STLC), and extends it with new constructions, modularly proving the strong normalization of the new calculuses. The reimplementing Pyrosome's main case-study using our tool instead. The formalization consists of a construction of STLC and a CPS calculus, as well as a translation pass from the former to the latter. The formalisation then extends both STLC and CPS with various constructionss, and extends the translation pass between the respective extensions.
Together, these two case-studies serve to show that the fundamentally basic ideas behind **Gemel** are powerful enough to scale up to all features provided by previous tools, as well as allowing one to to extend an independent formalisation after the fact, which is unique to our framework.

1.5 Summary and future work

We contribute a ready-to-use tool to easily extend past formalisation into new ones. That tool still exposes some limitations and a potential for further improvements which are discussed in the future works (Section 8), but is fairly complete in its original objective, as shown by our case-studies. A good next step would be to make the tool mature enough to be used in real world cases such as Lean's computer science library CSLib (Barrett et al. [2026]) , where e.g various type-systems formalisation could benefit from the tool.

2 Partial mappings

An initial idea for achieving modularity in our system was to rely on an "à la carte" encoding similar to Rocq à la Carte's, i.e rely on a notion of "feature functors" that can be composed to construct new inductive types.

```
inductive ExpVar (exp : Type) where
  | var : Nat → ExpVar exp

inductive ExpLam (exp : Type) where
  | app : exp → exp → ExpLam exp
  | lam : exp → ExpLam exp

inductive Exp₁ where
  | expvar : ExpVar Exp₁ → Exp₁
  | explam : ExpLam Exp₁ → Exp₁
```

Here, `ExpVar` and `ExpLam`, referred to as "feature functors", are parametrised by a type `exp`, and can be used to instantiate an inductive type with the desired features, `Exp₁` is such an example. Functions over types defined using feature functors can then be defined by first constructing said functions over the various feature functors, and composing them in an appropriate recursive function for the composed type:

```
def ExpLam.count (f : exp → Nat) : ExpLam exp → Nat
  | app e₁ e₂ => (f e₁) + (f e₂)
  | lam e => f e

def ExpVar.count (f : exp → Nat) : ExpVar exp → Nat
  | var _ => 1

def Exp₁.count : Exp₁ → Nat -- Error: Failed to show termination
  | expvar v => ExpVar.count Exp₁.count v
  | explam v => ExpLam.count Exp₁.count v
```

However, there is two issues with doing this. First, this does not allow for extending a formalisation after the fact, in order for a formalisation to be extended, it needs to be written using feature functors to start with, which is a uncommon design pattern which adds a fair bit of verbosity. The second issue is a technical one, and fairly specific to Lean, which is the language we chose to use for this work. The previous code would work in Rocq, but throws a non-termination error in Lean. Indeed, functions in proof-assistants are required to be total for the sake of ensuring the soundness of the type-system (i.e the inability to prove `False`). Every popular proof-assistant ensures this restriction differently (this is discussed more in depth in Section 4.2), but an important divergence between Rocq and Lean is that Rocq manages what is referred to as beta-iota cuts, i.e the ability to reduce function calls in which a recursive call might be nested in (e.g `ExpVar.count` in the `ExpVar.count Exp₁.count v` recursive occurence) in order to prove the recursive function being defined (here `Exp₁.count`) is indeed only called recursively on strictly smaller subterms, thus ensuring structural recursion. This feature

is paramount to Forster and Stark [2020]'s method for producing modular code, and cannot be easily replicated in Lean.

One core objective of **Gemel** is the ability to extend a formalisation or program after the fact, no matter the shape it may have been given by a former implementor. In order to do so, we shift our focus from encodings "à la Carte" to making clever use of the existing metaprogramming APIs provided by Lean in order to achieve our goal of modularity. In particular, we want to be able to translate any definition or theorem written about one inductive type into one that instead mentions an extended version of said type. We refer to this translation function, written onwards as $[\![\_]\!]$, as a **partial mapping** AA: I hadn't taken a second to think about this until now, but this is a terribly bad name ? Partial maps already exist and mean a completely different thing ?. Said mapping is referred to as partial because a term may get translated into something which contains holes, also known as metavariables. The reason why such holes may appear becomes more apparent in the next section. A partial mapping is dependent on a **mapping context**. This context maps some constants in the global environment to new terms. Using this mapping context, the algorithm for partial maps is straightforward: when partially mapping a constant present in the mapping context, map said constant to the term it maps to. Otherwise, translate terms structurally (e.g $[\![\lambda a : A.t]\!] \coloneqq \lambda a : [\![A]\!].[\![t]\!]$). The full presentation about how such partial mappings are implemented are described in Appendix A.

3 Inductive Types

We hereon make use of our system of partial maps to provide users with ways to modularly extend previously defined programs and formalizations. From a user perspective, a Lean program mainly consists of defining new inductive types, definitions and theorems. This section focuses on how **Gemel** allows users to extend inductive definitions through addition of new constructors.

3.1 What is an inductive type

First, let us look at what an inductive type is. We will not focus on the semantic meaning of inductive types so much as their syntax here, since what we care about is exposing new syntax to users. The general intuition of an inductive type is that it is a regular datatype, of which terms can be constructed by using recursors, and which can be eliminated through the use of a recursor, i.e an induction principle.
Inductive types are shaped as follows:

$$\textbf{inductive}\ \text{Ind}\ \overrightarrow{(p : P)} : \overrightarrow{(\imath : I)} \to \texttt{Type}\ \textbf{where}$$
$$\mid \textbf{ctor}_1 : \overrightarrow{(a_1 : A_1)} \to I\ \vec{p}\ \vec{d_1}$$
$$\ldots$$
$$\mid \textbf{ctor}_n : \overrightarrow{(a_n : A_n)} \to I\ \vec{p}\ \overrightarrow{d_n}$$

An inductive type is composed of a number of different components: A list of parameters $\overrightarrow{(p : P)}$ which are uniform in that they stay the same in every occurrence of the type in its constructors, a telescope of indices $\overrightarrow{(\imath : I)}$ that may vary in each occurrence, and a list of constructors for this type. Each constructor contains a list of fields living

over the context formed by the parameters, and instantiates the indices of the inductive type they inhabit in a context containing those fields. Basic examples of inductive types include the natural numbers, list, and vectors (i.e list indexed by their lengths):

```
inductive Nat : Type where
  | Z : Nat
  | S : Nat → Nat
```

```
inductive Vec (A : Type) : Nat → Type where
  | nil     : Vec A Z
  | cons {n}: A → Vec A n → Vec A (S n)
```

`Nat` is an inductive type with no parameters and no indices, as well as two constructors `Z` and `S`, the first one containing no field and the second containing one recursive occurrence of `Nat` as its sole field. In comparison, `Vec` is an inductive type which has one parameter `A`, is indexed by a natural number, and has two constructors. The former has no field and instantiate its index at 0, the second has 3 fields with one recursive one. `Vec A n` can be interpreted as the type of lists of `A` of size `n`.

To each inductive type is associated a recursor, i.e a way to recursively eliminate a term of that type. For example, the recursor for `Nat` has the following type:
`Nat.rec` : $(\mathbf{motive} : \texttt{Nat} \to \texttt{Type}) \to \mathbf{motive}\ \texttt{Z} \to ((n : \texttt{Nat}) \to \mathbf{motive}\ n \to \mathbf{motive}\ (\texttt{S}\ n)) \to (t : \texttt{Nat}) \to \mathbf{motive}\ t$
The existence of that recursor can be interpreted as the statement that, given a predicate **motive** on natural numbers, an instance of that predicate on 0, and for every n, an instance of **motive** $(\texttt{S}\ n)$ given **motive** n, that predicate holds for any natural number t. This corresponds exactly to the regular induction principle for natural numbers.

3.2 Our approach towards extending inductive types

With the general structure of an inductive type in mind, we can now consider ways in which one may extend an inductive type into another.
There are morally two ways to extend an inductive type, which we refer to as vertical and horizontal extensions. The former consists of adding new constructors to an inductive type, the latter of adding new parameters or indices to it. Most modular systems only allow for doing vertical extensions, as it is both the easiest one to justify and implement well, and arguably the most useful one. Currently, **Gemel** only manages vertical extensions, we discuss in our second case-study (Section 6.2) how some horizontal extensionality can be constructed in the system nonetheless with careful use of dependent types.

Gemel provides a new Lean command which, given an inductive and new constructors, constructs another inductive type which extends the original one with these new constructors, adding the relevant mappings between from the first to the second. The syntax is as follows:

```
mod inductive <new inductive> extends <old inductive> where
  <new constructors>
```

Consider the following example: A user defines a notion of a variable type `Var`, which only has one constructor consisting of a natural number.

```
inductive Var where
  | var : Nat → Var
```

This representation for variables is common in both imperative and functional intermediate representations of programs. After producing an extensive API for this type, one may want to use this notion of variables as part of another type. Consider a new type `Term` which corresponds to that of a lambda-calculus. Such a calculus is constructed using variables, lambdas and applications. Such a type can be defined by extending the variable type as follows:

```
mod inductive Term extends Var where
  | lam : Term → Term
  | app : Term → Term → Term
```

Similarly, one may want to extend the lambda-calculus with other constructs, such as booleans. Since `Term` is also just an inductive type, it can itself be extended with new constructions:

```
mod inductive BoolTerm extends Term where
  | true  : BoolTerm
  | false : BoolTerm
  --- if     c    then    a    else    b
  | ite : BoolTerm → BoolTerm → BoolTerm → BoolTerm

-- λ b => if b then false else true
#check lam (ite (var 0) false true) -- BoolTerm
```

Internally, after constructing this new type, **Gemel** adds the relevant mappings from the old type to the new one in the mapping context: Consider an inductive `A` of parameters $\overrightarrow{(p : P)}$, indices $\overrightarrow{(\imath : I)}$ and constructors $\overrightarrow{\mathbf{ctor}}$, as well as another type `B` with the same parameters and indices, as well as the same constructors + a new constructor $\mathbf{newctor}$, we can then map the type A to B and respectively every constructor of A to B's. The last missing piece to this translation is the recursor.

A recursor has the shape:
$\text{A.rec} : \overrightarrow{(p : P)} \to \left(\mathbf{motive} : \overrightarrow{(\imath : I)} \to A\ \vec{p}\ \vec{\imath} \to \texttt{Type}\right) \to$
$\overrightarrow{(\mathbf{minor} : \ldots \to \mathbf{motive}\ (\mathbf{ctor}\ \ \ldots))} \to \overrightarrow{(\imath : I)} \to (a : A\ \vec{p}\ \vec{\imath}) \to \mathbf{motive}\ \vec{\imath}\ a$
where for every constructor of the type, there is a `minor` covering the (potentially recursive) case associated to it.
`B`'s recursor is very similar, except it contains an additional minor for the `newctor`'s case.
Gemel partially maps any application with head `A.rec` into `B.rec` by partially mapping each argument (i.e the parameters, motive, minors and major), and adding a metavariable hole for the minor corresponding to `newctor`:

$$\left[\!\left[\text{A.rec}\ \vec{p}\ \mathbf{motive}\ \overrightarrow{\mathbf{minor}}\ \vec{\imath}\ a\right]\!\right] := \text{B.rec}\ \overrightarrow{[\![p]\!]}\ [\![\mathbf{motive}]\!]\ \overrightarrow{[\![\mathbf{minor}]\!]}\ ?m\ \overrightarrow{[\![\imath]\!]}\ [\![a]\!]$$

This mapping is automatically produced and added to the mapping context upon

constructing a `mod inductive`. Consider the previous example of `Var` and `Term`, their respective recursors are as follows:

```
Var.rec  : (motive : Var  → Type) → ((a : Nat) → motive (var a)) →
  (t : Var) → motive t
Term.rec : (motive : Term → Type) → ((a : Nat) → motive (var a)) →
  ((a : Term) → motive a → motive (lam a)) →
  ((f t : Term) → motive f → motive t → motive (app f t)) →
  (t : Term) → motive t
```

We see that the motive, existing minor for `var` and the major `t` can be translated into a `Term.rec` call by simply adding new holes for the `lam` and `app` minors. The translation goes as follows:
`⟦Var.rec motive pvar t⟧ => Term.rec ⟦ motive ⟧ ⟦ pvar ⟧ ?m1 ?m2 ⟦ t ⟧`
Note that metavariable holes such as `?m1` and `?m2` cannot, in general, be resolved automatically by Lean, and are instead meant to be resolved by the user, the next section on definitions extensions discusses the interface provided by **Gemel** for that purpose in more details.

Additionally to the primitives surrounding an inductive type, Lean also automatically produces more than a dozen auxiliary declarations, meant to make the automation and user-experience friendlier. For example, upon defining an inductive type `Foo`, Lean will, for every constructor `ctor`, define a helper lemma `Foo.ctor.inj` which proves the injectivity property of said constructor. In order to avoid making users define mappings between auxiliary declarations produced by the original and the extended type by hand, **Gemel** automatically generates said mappings automatically (e.g `Var.var.inj` gets automatically mapped to `Term.var.inj` in the previous example).

4 Definition extensions

Our framework now allows for inductive types to be extended through the addition of new constructors. This section focused on how declarations (i.e definitions and theorems) can be extended and partially mapped correctly.
Going back to the previous example of `Term` extending `Var`, consider a function `Var.repr`, mapping a term of type `Var` to a string:

```
def Var.repr : Var → String
  | var n => s!"#{n}"
```

Currently, a user would need to redo the `var` case in order to define a `Term.repr` function, e.g as follows:

```
def Term.repr : Term → String
  | var x => s!"#{x}"
  | app f x => s!"{Term.repr f} {Term.repr x}"
  | lam f => s!"λ {Term.repr f}"
```

This however leads to a duplication of the `var` case, which in turns lead to a burden of maintenance. If, for example, the user was to change how variables are pretty-printed in the `Var.var` case, this change would have to be copy-pasted for the `Term.var` case, which would be hard to track in general. Instead, **Gemel** introduces a new `mod def` syntax to allow users to reuse existing branches in a previously defined function, and only require them to fill-in the holes needed to go from a definition over one type to a definition of the same shape over an extension of said type. This syntax allows to rewrite `Term.repr` as follows:

```
mod def Term.repr extends Var.repr where
  extend with
  | app f x => s!"{Term.repr f} {Term.repr x}"
  | lam f => s!"λ {Term.repr f}"
```

Now, any change done to `Var.repr` would also change downstream of it for `Term.repr`.

The general elaboration process of that syntax is as follows. Given a declaration (e.g `Var.repr`), one may partially map its body into a function over the extended type (here `Term`). Once that it done, users may be asked to fill in any remaining holes in the translated body to construct a well-formed declaration. Once that is done, the new declaration (here `Term.repr`) can be safely added to the global environment of declarations, and a partial mapping from `Var.repr` to `Term.repr` can be added to the mapping context.

In practice, implementing all of this in Lean has exposed some complications which necessitates careful design decisions, in particular with regards to how pattern-matching and recursion are handled by the system. We discuss these two complications in the following sections.

4.1 Extending pattern-matches

Pattern-matching is a ubiquitous feature of modern functional languages, and proof assistants make no exception of that. In Rocq, matches are a primitive operation made explicit in the abstract syntax. In Agda, functions are defined as case trees, allowing users to branch on terms, similarly to regular matches.
Lean diverges from the tradition. For users, it may appear as though Lean does have match expressions: they are part of the concrete syntax and get appropriately pretty-printed when looking back at the abstract syntax. In practice, Lean's elaborates pattern-matches down to the necessary inductive recursors, following technics described in Goguen et al. [2006]. Take for example a function of the form

```
def is_var_zero : Var → Bool
  | var 0     => true
  | var (n+1) => false
```

The shape of the match get elaborated into an auxiliary function:

```
is_var_zero.match_1 : (motive : Var → Type) → (x : Var) →
(Unit → motive (var 0)) → ((n : Nat) → motive (var (n + 1))) → motive x
```

The function is then applied with the right arguments to explicit 1. the return type of the match-here called the `motive`- the discriminant -here `x`- and the right-hand-side of each branch:

```
def is_var_zero : Var → Bool := fun x =>
  is_var_zero.match_1 (fun _ => Bool) x (fun _ => true) (fun _ => false)
```

Note that a `Unit` argument appears in the `var 0` branch. This happens because Lean compiled code is call-by-value, which justifies lazily computing the branches of a given match.

In practice however, the pretty-printer recognises when an application has a match expression as its head, and thus prints it back to the user as a `match`.

When it comes to extending matches, we are presented with two options. The first one consists of remarking that matches are simply encoded using recursors. As such, we may simply partially map a given matcher and ask users to fill in the holes in it. Another one would be to remark that extending a match amounts to adding new branches to it such that it covers the relevant new constructors in the extended inductive type. As such, we may ask users to write down the relevant branches, in a syntax similar to how normal matches are written. The former is easier to implement, it however noticeably burdens the user-experience. The second is more complex to implement, but makes the user-experience be on par with that or regular definitions. After initially implementing the first option, we have converged towards relying on the second instead.

The first option of asking users to fill in the proof-holes of the extended recursors that constructs a match is unreasonable, as it exposes far too many internal details. The definition of a given match can quickly become complex if it matches on multiple elements, needs to generalize some variables or contains some proofs of equality between the thing matched on and a given constructor in a branch. Extending any of this would make an unreadable mess for the users and is too detached from what a user would do had he just written this as a normal definition. Consider the case of `is_var_zero` being extended to `Term`. For both the `app` and `lam` case, the result is the same (i.e `false`). Producing this extension would require both extending the implementation of `match_1` (by adding a branch for `(t : Term) → motive (lam t)` and `(f t : Term) → motive (app f t)`), and adding the right arguments for the right-hand-side of said new branches (i.e `fun t => false` and `fun f t => false`). This means, in this specific example, that a user would need to fill in 4 holes, half of them having a non-trivial shape which mentions an arbitrary `motive` that did not appear in the concrete syntax of the original definition.

The second option turns out to be the most ergonomic, although it makes the implementation work harder. In order to accomodate for this feature, when partially mapping the term of a given declaration, any application whose head is a matcher gets translated into a metavariable hole, saving the original shape of the expression along the way. When trying to solve that metavariable, the original shape of the matcher is reconstructed, similarly to how the pretty-printer handles this expression. We then elaborate the new match branches provided by the user, and re-elaborate this into a new match-expression.

Note that the implementation does **not** do any anti-quotations (i.e it does not construct concrete syntax from the original abstract syntax), and instead manipulates both the abstract syntax of the old matcher and the concrete syntax of the new branches in parallel, using Lean's existing internals for elaborating matchers. The end-result is a legible syntax for extending past declarations.
Take the example of `is_var_zero` getting extended to `Term`:

```
def Term.is_var_zero extends is_var_zero where
  extend with
    | app _ _ => false
    | lam _   => false
```

First, the extension system detects that `is_var_zero.match_1 (fun _ => Bool) (fun _ => true) (fun_ => false)` is a matcher application and stores the information about its shape (i.e what the motive, discriminants and existing branches are). All of this information gets partially mapped to `Term` rather than the original `Var` (e.g `Var.var` become `Term.var` in the left-hand side of each branch). Then, the branches provided by the user (i.e `| app _ => false` and `| lam _ _ => false`) get elaborated as new branches to be added to the previous ones. This bundle of data is then passed out to the existing API Lean uses to elaborate normal pattern-matches, and produces a new match application that the old one then gets translated to.

4.2 Extending recursive functions

Dependently-typed proof-assistants like Rocq implement recursive functions using fix-point constructs that are primitive to the abstract syntax, coupled with syntactic criterias for making sure recursive functions are well-founded. Lean, on the other hand, does not have such constructs. Instead, when a recursive function is defined, Lean tries to elaborate the function either into a structurally recursive term, using the recursor of whatever term decreases structurally in the recursive calls to write the function (à la McBride [1999]), or tries to prove the function be well-founded, morally recursing over the accessibility predicate.

Because of this, directly partially mapping the body of a recursive definition is not great, since it exposes internal encodings to the user, and asks one to manipulate those directly to extend the definition. Thankfully, Lean usually provides auxiliary lemmas that exhibit the original shape of the function that was defined. One of them, `foo.eq_def`, is a theorem of the form:

```
(a₁ : A₁) → ... → (aₙ : Aₙ)  → foo a₁ ... aₙ = <foo's definition>
```

As such, rather than use the original body of a definition, we can instead partial map the right-hand side of a function's `eq_def` whenever available, and then reuse the existing elaboration APIs Lean provides to translate such syntacticaly recursive functions to structurally recursive or well-founded functions. This in effect means a user can adapt the termination measure of an extended definition, relative to the original's, a feature no other extension system provides to our knowledge. Said feature is of particular impor-

tance when extending a non-recursive type to a recursive one, as is the case with the extension from `Var.repr` to `Term.repr`.

In particular, this allows us turn an originally non-recursive function into a recursive one if needed, as is the case with `Term.repr` extending `Var.repr`, or even change the termination measure that was originally used to adapt it to the new extended function.

4.3 Putting it all together

In practice, being able to change the proof of termination of an extended function compared to the original provides us with much more modularity, and is a feature we have not seen in any other project of the sort.
Furthermore, having matches be handled specially provides a clear distinction between the holes generated for matches and those generated by explicit uses of recursors, which usually only appear in proof terms generated by tactic calls such as `induction` and `cases`. branches. The complete syntax for `mod def` ends up as the following:

```
mod def <new function name> extends <old function name> where
  <match extension>
finally
  <tactics>
<termination-information>
```

This in essence means our syntax for modular definitions offers a clear separation of concerns between proof holes, solved by tactics, and non-proof holes, solved by completing match This sort of distinction is not new. Indeed, `where finally` is already a feature for definitions in Lean, and can be used to fill-in proof holes after the fact:

```
def Vec.tl (v : Vec α (S n)) : Vec α n :=
  match h : S n, v with
    | Z, nil => nomatch h
    | S k, cons _ tl => (show n = k from ?_) ▸ tl
                  --rewrites the type of `tl` from `Vec α k` to `Vec α n`
  where
    finally
    injection h
```

Similarly, Rocq's `Equations` (Sozeau and Mangin [2019]) syntax provide an "Obligations" system which serve a similar purpose. We thus adopt a similar syntax to Lean's normal `def` declarations, by asking for matchers to be completed first, before asking proof holes to be solved in a `finally` block. The `finally` block mainly only appears when extending theorems rather than definitions. Consider the following theorem:

```
theorem is_var_zero_eq (t : Var) (h : is_var_zero t) : t = var 0 := by
  cases t with
  | var n =>
    cases n with
    | zero => rfl
    | succ _ => nomatch h
```

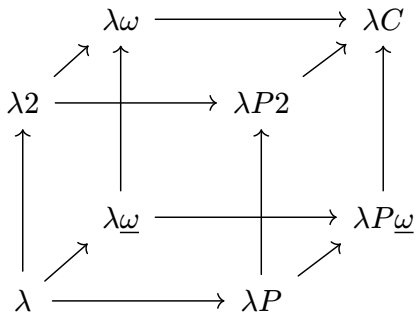
Extending it for `Term` will add two new proof holes originating from the `cases t`. These are given by the user in the `finally` block as follows:

```
mod def Term.is_var_zero_eq extends is_var_zero_eq where
  finally
  case lam _   => nomatch h
  case app _ _ => nomatch h
```

Termination information such as `termination_by` or `decreasing_by` can be additionally provided if needed to help with producing a well-founded recursive function. For example, the previous `Term.repr` function could have had an explicitly given annotation such as `termination_by structural t => t`, indicating that the function is structurally recursive over it's argument `t : Term`. In practice, and as is the case for the regular Lean definition, most functions' termination information is trivial enough to be inferred by the Lean, and thus does not require additional user-input.

5 Making extensions modular: modular merges

The current set-up allows one to extend previous definitions iteratively, though one may argue these extensions are not strictly "modular". In particular, one can currently only construct a declaration as an extension of a single other declaration rather than multiple. This, in particular, means one isn't able to compose different extensions. Take the example of the Barendregt lambda-cube (Barendregt [1991]):



The various vertices of the cube can be seen as various extensions of the initial vertex corresponding to STLC. However, an important feature of this cube is that all extensions of λ can be written as mergings of its 3 adjactent vertices, namely λP, $\lambda 2$ and $\lambda\underline{\omega}$ (e.g $\lambda P2$ corresponds simply to the merging of $\lambda 2$ and λP). If one wanted to formalise each vertex of the cube before **Gemel**, they would need to write down 8 different formalisations. With our current framework, they need only write down 1 formalisation and 7 extensions of that base formalisation. If our system was truly modular however, one would only need to write down the base formalisation and the 3 adjacent extensions, the rest being simply generated by merging the adjacent extensions. To make this possible, we extend **Gemel**'s handling of inductive types and definitions extensions and describe a way to make them composable, thus achieving true modularity. In practice, from a user perspective, all that gets added is the ability to define a `mod inductive` or `mod def` that `extends` more than one single declaration.

5.1 Inductive modularity

Extending `mod inductive` to be able to extend more than one inductive type is a fairly trivial task. Rather than only taking the constructors of one inductive, users may take constructors from multiple types. The mapping context will then map every old type to the new one, every old constructor from each old type to the relevant new ones. Consider the example of `BoolTerm` extending `Term` in Section 3.2, one may also extend `Term` to handle natural numbers, and merge the two extensions as follows:

```
mod inductive NatTerm extends Term where
  | zero : NatTerm
  | succ : NatTerm → NatTerm
  --   match n with | 0 => P0 | n+1 => Pn n
  | natmatch : NatTerm → NatTerm → NatTerm → NatTerm

mod inductive BoolNatTerm extends BoolTerm, NatTerm
```

The generated inductive `BoolNatTerm` will contain all of the basic `Term` constructors, as well as the additional ones provided by `BoolTerm` and `NatTerm`, will map the old inductives' constructions to the appropriate constructions of the new one.

5.2 Definition modularity

Consider the previous examples of defining `is_var_zero` and `is_var_zero_eq` to `BoolNatTerm`, consider the two declarations have already been extended to `BoolTerm` and `NatTerm`:

```
mod def BoolTerm.is_var_zero extends Term.is_var_zero where
  extend with
    | true | false | ite _ _ _ => false

mod def NatTerm.is_var_zero extends Term.is_var_zero where
  extend with
    | zero | succ _ | natmatch _ _ _ => false
```

We can partially map both declarations into functions talking about `BoolNatTerm`. Both of them will be partial in that the pattern-match will be incomplete in both cases (e.g the cases for `zero` will be missing from the partial mapping of `BoolTerm.is_var_zero`). However, both declarations will have the same shape modulo having different holes in different places. We make use of that and construct an algorithm that syntactically merges two expressions together, and throws an error if their shape diverges. In practice, in the case of `is_var_zero`, this means the respective matches get merged correctly, meaning there is no need for the user to add anymore information in order to merge the two declarations:

```
-- No `extend with` block needed to complete the match
mod def BoolNatTerm.is_var_zero
  extends BoolTerm.is_var_zero, NatTerm.is_var_zero
```

Similarly, the proof for `is_var_zero_eq` need not anymore information after merging both `BoolTerm.is_var_zero_eq` and `NatTerm.is_var_zero_eq`:

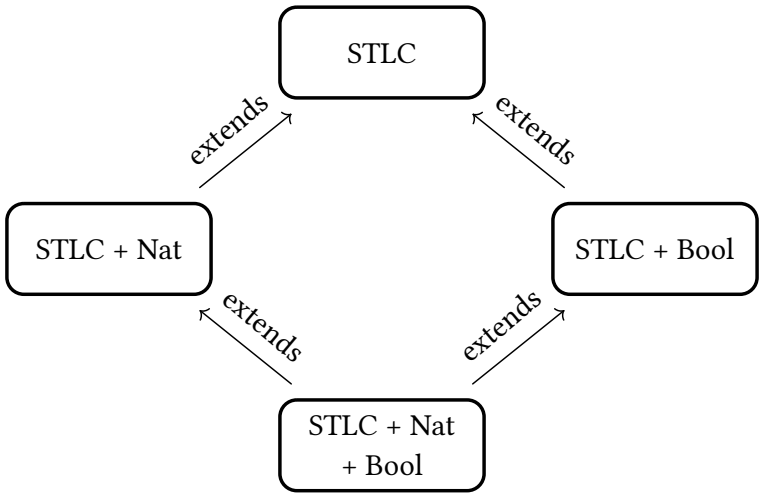
```
mod def BoolTerm.is_var_zero_eq extends Term.is_var_zero_eq where finally
  ... -- proof omitted

mod def NatTerm.is_var_zero_eq extends Term.is_var_zero_eq where finally
  ... -- proof omitted

-- no `finally` block needed
mod def BoolNatTerm.is_var_zero_eq
  extends BoolTerm.is_var_zero_eq, NatTerm.is_var_zero _eq
```

6 Case studies:

6.1 Strong normalisation of the Simply Typed Lambda Calculus



In order to iterate on this implementation and ensure its usability, we studied the case of taking an existing implementation of STLC[2] which prove the strong normalization property of the system, and extended it with additional constructors, allowing us to modularly prove SN for the extended system. In particular, the original normalization was **not** built with modularity in mind, and was still extendable after the fact. The original formalisation is extrinsically typed, and follows the usual proof of normalisation using a logical relation.

[2]https://github.com/amarmaduke/lean-stlc

```
inductive Ty : Type where
| arrow : Ty → Ty → Ty
```

```
inductive Term where
| var : Nat → Term
| app : Term → Term → Term
| lam : Term → Term
```

```
inductive Typing : List Ty → Term → Ty → Type where
  | tyvar : Γ[n]? = some A → Typing Γ (var n) A
  | tyapp : Typing Γ f (arr A B) → Typing Γ t A → Typing Γ (app f t) B
  | tylam : Typing (A::Γ) t B → Typing Γ (lam t) B
```

We extend this base definition to add natural numbers:

```
mod inductive NatTerm.Ty
  extends Ty where
  | nat
```

```
mod inductive NatTerm.Term
  extends Term where
  | zero   : Term
  | succ   : Term → Term
  | natRec : Term → Term → Term → Term
```

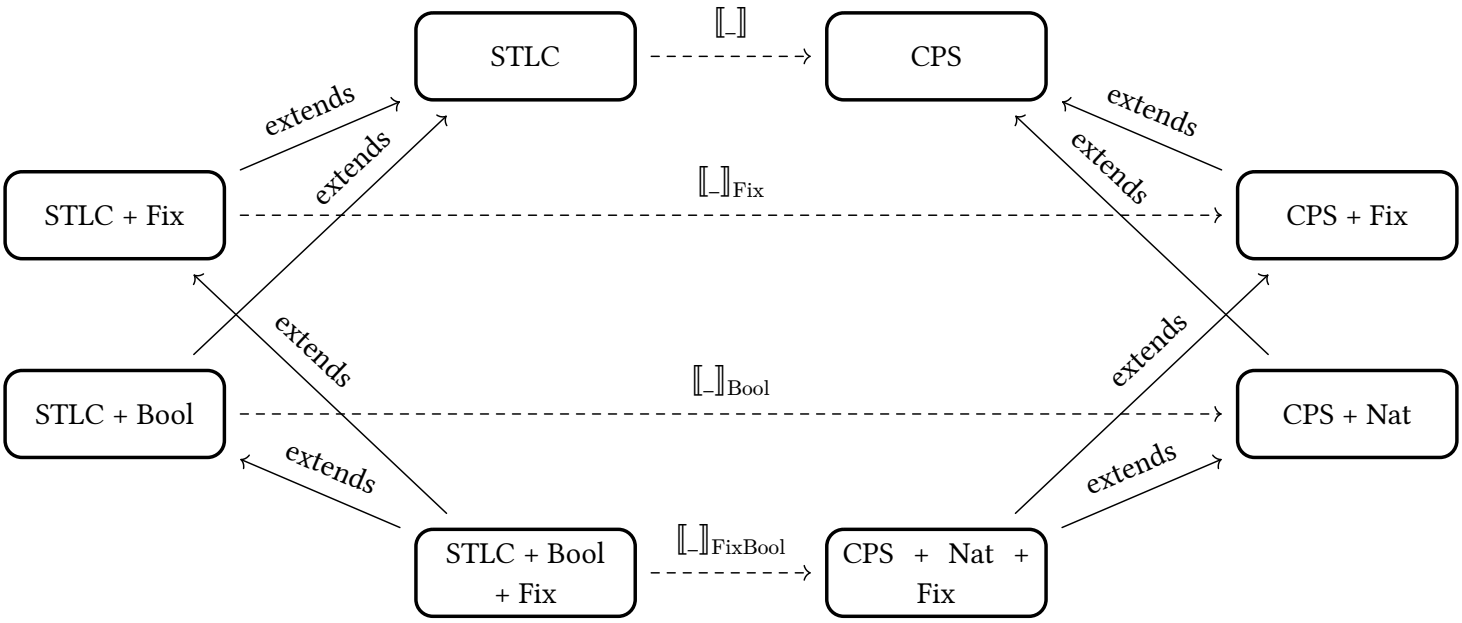
```
mod inductive NatTerm.Typing extends Typing where
  | zero   : Typing Γ zero nat
  | succ   : Typing Γ n nat → Typing Γ (succ n) .nat
  | natRec : Typing Γ n nat → Typing Γ P0 A
    → Typing Γ PS (arr nat (arr A A)) → Typing Γ (natRec n P0 PS) A
```

The total formalisation contains 12 inductive types as well as 92 definitions/theorems. Almost all of these were extended with no issue. The sole limiting factor in reaching the end of the formalisation relying only on extensions was with regards of the logical relation used in the original formalisation to prove strong normalisation: their initial formulation was too weak to prove SN on a system extended with natural numbers, and the theorems surrounding it relied heavily on definitional equalities of `LR` that could not be recovered after partially mapping it to something strong enough for our use-case. As such, the fundamental lemma itself had to be written as a normal lean `def` rather than an extension. A possible solution to circumventing such issues, based on the idea of removing the reliance of some definitional equalities on a given term, is discussed in the future works.

To showcase the capacity to merge separate extensions, we produce another extension of STLC containing primitives for booleans, namely a boolean constructor in `Ty`, and term constructors for `true`, `false` and `if-then-else`. We then successfully construct a formalisation of STLC with both natural numbers and booleans by merging the two previous formalisations, thus proving the strong normalization of "STLC + Bool + Nat" fully modularly.

AA: Maybe give some approximation of the numbers of lines saved ?

6.2 STLC to CPS translation



To showcase the capabilities of our framework, and provide a point of comparison with another modular system, we reproduce the central case study shown in Pyrosome's (Jamner et al. [2025]) paper. This case study introduces

AA: Explain the challenges of intrinsic verification, how Pyrosome's GAT was circumvented with indexed inductives and quotients. Explain how parts of the ability to do horizontal extensions was recovered here as well. Also explain that this presentation is intrinsic whereas the previous case study's was extrinsic.

7 Related works

We compare our approach with the recent literature with a special focus on approaches that adapt Data Types à la Carte to proof assistants.

Data Types à la Carte TODO

"Meta-Theory à la Carte" (Delaware et al. [2013]) and "Pyrosome" (Jamner et al. [2025]) base their modularity on internal encodings of types (namely, impredicative church-encodings in the former, and Generalized Algebraic Theories in the latter). In both cases, the constructions are inefficient, incapable of extending previously user-defined inductive types (*i.e.,* expecting users to rely on the aforementioned encodings from the ground up), and expose the underlying internals of the encodings to the user. Having to deal with encodings of types rather than types adds a heavy burden in particular for new users interested in program verification. The lesson to include inductive data-types natively has been learned early by popular ITPs (namely Rocq and Lean), who at first had no primitive notions of inductive types in their systems and then resorted to add those. Nowadays, most systems usually possess a syntactic notion of (co-)inductive types, and justify the ability to define these via some classes of models, allowing users to work with the abstractions these types provide, rather than with their encoding in said models. The

only popular system that still relies on encodings to define such types, and manages to hide their implementation details well, is Isabelle/HOL.

On the other hand, Rocq à la Carte (Forster and Stark [2020]) relies on the metaprogramming capabilities offered by the MetaRocq Project (Sozeau et al. [2020]) to allow users to construct new inductive types and functions by merging other inductive types, and/or adding new constructors. New functions on a "merged" datatype can then be constructed by merging past functions. The metaprogram then uses the given piece of information to reconstruct a new inductive type, and new functions, based on the information given by the user. While great for extending constructions "vertically" (*i.e.,* by adding constructors to a type), this system does not allow for horizontal extensions (*i.e.,* extending the type signature of inductive types and their constructors). Furthermore, this approach has been hindered in the past by the lack of good metaprogramming frameworks in ITPs.

Rocqet TODO

None of these systems, independently of whether they use encodings or the metaprogramming, handle all of the type-system of ITPs they are implemented for (*e.g.,* none handle co-inductive types), or allow users to extend past formalizations "after the fact". Instead, formalizations have to be built from the ground up with the expectation that they will be extended with a specific framework in mind, making them much less useful for real-world uses.

AA: TODO talk in details about Rocquet, Datatypes à la Carte, Meta-Theory à la Carte, Pyrosome, Rocq à la Carte

8 Future work

- Horizontal extensions
- ETT to ITT translation to circumvent divergences in reduction behaviour between a term and its translation
- Zipper-like structure on expressions to traverse "up" and generalize a goal "after the fact"

References

AWS (2026) Cedar Specification, 2026, https://github.com/cedar-policy/cedar-spec.

Barendregt, H. (1991) An Introduction to Generalized Type Systems, *Journal of Functional Programming*. 1991;1: 125–154., doi:10.1017/S0956796800020025.

Barrett, C., Chaudhuri, S., Montesi, F., et al. (2026) CSLib: The Lean Computer Science Library, *arXiv e-prints*. advance online publication February 2026;arXiv:2602.04846., doi:10.48550/arXiv.2602.04846.

Delaware, B., S. Oliveira, B.C. d. and Schrijvers, T. (2013) Meta-theory à la carte, *SIGPLAN Not.* 2013;48(1): 207–218., doi:10.1145/2480359.2429094.

Forster, Y. and Stark, K. (2020) Coq à la carte: a practical approach to modular syntax with binders, In: Blanchette, J. and Hritcu, C. (eds.). *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*, ACM 2020: pp. 186–200, doi:10.1145/3372885.3373817.

Goguen, H., McBride, C. and McKinna, J. (2006) Eliminating Dependent Pattern Matching, In: Futatsugi, K., Jouannaud, J.P. and Meseguer, J. (eds.). *Algebra, Meaning, And Computation, Essays Dedicated to Joseph A. Goguen on the Occasion of His 65th Birthday*. Vol 4060. Lecture Notes in Computer Science, Springer 2006: pp. 521–540, doi:10.1007/11780274_27.

Jamner, D., Kammer, G., Nag, R. and Chlipala, A. (2025) Pyrosome: Verified Compilation for Modular Metatheory, *Proc ACM Program Lang.* 2025;9(OOPSLA2)., doi:10.1145/3763052.

Kovács, A. (2020) Elaboration with first-class implicit function types, *Proc ACM Program Lang.* 2020;4(ICFP)., doi:10.1145/3408983.

Leroy, X. (2009) Formal verification of a realistic compiler, *Communications of the ACM*. 2009;52(7): 107–115., http://xavierleroy.org/publi/compcert-CACM.pdf.

Lyons, A., McLeod, K., Klein, G., et al. (2025) seL4/seL4: seL4 14.0.0, December 2025, doi:10.5281/zenodo.17904671.

The mathlib Community (2020) The Lean Mathematical Library, In. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020, Association for Computing Machinery, New Orleans, LA, USA 2020: pp. 367–381, doi:10.1145/3372885.3373824.

McBride, C. (1999) *Dependently Typed Functional Programs and Their Proofs*, phdthesis, 1999.

Oliveira, B.C. d. S. and Cook, W.R. (2012) Extensibility for the Masses, In: Noble, J. (ed.). *ECOOP 2012 – Object-Oriented Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg 2012: pp. 2–27.

Sozeau, M. and Mangin, C. (2019) Equations Reloaded: High-Level Dependently-Typed Functional Programming and Proving in Coq, *Proc ACM Program Lang.* 2019;3(ICFP)., doi:10.1145/3341690.

Sozeau, M., Anand, A., Boulier, S., et al. (2020) The MetaCoq Project, *Journal of Automated Reasoning*. advance online publication 2020., doi:10.1007/s10817-019-09540-0.

Sozeau, M., Forster, Y., Lennon-Bertrand, M., Nielsen, J., Tabareau, N. and Winterhalter, T. (2025) Correct and Complete Type Checking and Certified Erasure for Coq, in Coq, *Journal of the ACM*. advance online publication January 2025., doi:10.1145/3706056.

Wadler, P. (1998) The expression problem, November 1998, https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt.

A Formal presentation of partial mappings

This section makes a partial attempt at justifying the implementation of partial mappings in our system, and why it can be trusted to make well-formed terms.

First, let's define the syntax we'll be using in this toy presentation. This is a basic dependently-typed system equiped with a predicative hierarchy of universes à la Russel, as well as constants and metavariables. For the sake of simplifying the presentation, we will assume constants cannot be unfolded (i.e we will only care about β/η-reduction, not δ-reduction). This, in particular, does not give good justification as to why recursors of inductive types may be extended safely.

$$
\begin{aligned}
\text{Terms} : t, f, A, B \coloneqq\ & x && \text{(variable)} \\
\mid\ & d && \text{(constant)} \\
\mid\ & m && \text{(metavariable)} \\
\mid\ & f\ t && \text{(application)} \\
\mid\ & \lambda(x : A) \mapsto t && \text{(abstraction)} \\
\mid\ & (x : A) \to B && (\Pi\text{-type}) \\
\mid\ & \texttt{Type}_i && \text{(universe)}
\end{aligned}
$$

Listing 1: Syntax of our type theory

The usual typing judgements one would expect apply. These judgement need to carry not only the usual variable context Γ, but also a global constants context Σ to handle the type of constants, similarly to Sozeau et al. [2025], as well as a metavariable context Θ that holds, for each metavariable, both the context and the type of said metavariable, similarly to Kovács [2020]. The idea is that constants may be mapped to some term containing "holes", i.e metavariables, allowing us to make the partial maps.

We define a judgement $\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$ encompassing the behaviour of the partial mapping in practice. This judgement carries the same contexts found in the aforementionned typing judgements, as well as a mapping context Φ, which maps constants to the bundling of a metavariable context and a term that may contain the metavariables present in the context it's bundled with. The judgement states tracks both the base variable context and a variable context for the partially mapped term, as well as the type of both the original term and the partially mapped one. The partial mapping acts structurally over the usual constructors of our type-theory, and relies on the mapping context to translate constants into their partial mapping. Furthermore, when partially mapping a constant, the metavariable context it carries is weakened/lifted, such that every metavariable lives in some telescope over translated variable context.

$\boxed{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'}$

(In global context Σ, mapping context Φ, metavariable context Θ, term t of type A in context Γ maps to term t' of type A' in context Δ)

AA: TODO fix spacing

$$\frac{}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash \mathtt{Type}_i \Rightarrow \mathtt{Type}_i : \mathtt{Type}_{i+1} \Rightarrow \mathtt{Type}_{i+1}}$$

$$\frac{(x : A) \in \Gamma \qquad (x : A') \in \Delta}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash x \Rightarrow x : A \Rightarrow A'}$$

$$\frac{((d : A) \mapsto (\Theta, t : A')) \in \Phi}{\Sigma \mid \Phi \mid \Uparrow_\Delta \Theta \mid \Gamma \Rightarrow \Delta \vdash d \Rightarrow t : A \Rightarrow A'}$$

$$\frac{\begin{array}{c}(m : (\Gamma_2, A)) \in \Theta \quad \Gamma_2 \subset \Gamma_1 \quad \Delta_2 \subset \Delta_1 \\ \Sigma \mid \Phi \mid \Theta \mid \Gamma_2 \Rightarrow \Delta_2 \vdash A \Rightarrow A' : \mathtt{Type}_i \Rightarrow \mathtt{Type}_i\end{array}}{\Sigma \mid \Phi \mid \Theta \mid \Gamma_1 \Rightarrow \Delta_1 \vdash m \Rightarrow m : A \Rightarrow A'}$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash f \Rightarrow f' : (x : A) \to B \Rightarrow (x : A') \to B' \\ \Sigma \mid \Phi \mid \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash f\ t \Rightarrow f\ t : B[x := t] \Rightarrow B'[x := t']} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash A \Rightarrow A' : \mathtt{Type}_i \Rightarrow \mathtt{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash B \Rightarrow B' : \mathtt{Type}_j \Rightarrow \mathtt{Type}_j)\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathtt{Type}_{\max(i,j)} \Rightarrow \mathtt{Type}_{\max(i,j)}} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathtt{Type}_i \Rightarrow \mathtt{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash t \Rightarrow t' : B \Rightarrow B')\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (\lambda(x : A) \mapsto t) \Rightarrow (\lambda(x : A') \mapsto t') : (x : A) \to B \Rightarrow (x : A') \to B'} \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing$$

Figure 1: Judgement rules for mappings

The data contained in this judgement is enough to ensure any partially mapped term to be well-typed. Note that this theorem critically relies on the fact that this system only uses

Theorem 1 Given a mapping context Φ, if for each $((d : A) \mapsto (\Theta, t : A')) \in \Phi$:

1. $\Sigma \mid \varepsilon \mid \varepsilon \vdash d : A$
2. $\Sigma \mid \Theta \mid \varepsilon \vdash t : A'$
3. $\Sigma \mid \Phi \mid \Theta \mid \varepsilon \Rightarrow \varepsilon \vdash A \Rightarrow A' : \mathtt{Type}_i \Rightarrow \mathtt{Type}_i$ for some i,

Then for all $\Gamma,\ t,\ A$ s.t $\Sigma \mid \Theta \mid \Gamma \vdash t : A$, there exists $\Delta,\ t',\ A'$ s.t $\Sigma \mid \Theta \mid \Delta \vdash t' : A'$ and $\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$

Proof: by induction on the typing judgement $\Sigma \mid \Gamma \vdash t : A$. AA: TODO: This theorem is false, actually 🥴